

Concert Booking - MEAN Stack

Problem Statement (reconstructed from the real Fresco/TCS mock screenshots)

Note

- To run the full stack application, execute the run command/script in the project.
- To test the full stack application, execute the test command/script.
- This application is used to book tickets for concerts and search for and view concert details.

FRONT-END: Angular

- This application uses Angular as the front-end. Fix the test case errors in the application. The test cases are written in Jasmine.

App Component

- Display all concerts and their details, and each concert should have a Book button.
- The concert details should come from the backend.
- If the user clicks the Book button, it should show a prompt or input field to get the number of tickets to be booked.
- After entering the number of tickets, the application should validate the input and then call the backend only for valid input.
- If tickets are booked successfully, show the alert message:

```
Tickets booked successfully
```

- If the user gives any strings, special characters, or blank spaces, show the alert message:

```
Input field should be valid
```

- If the number of available tickets is zero but the user clicks the Book button, show the alert message:

```
The tickets are sold out
```

- If the user enters a higher number than the available tickets, show the alert message:

```
Sorry, we only have ${available_tickets} tickets available. Your request for  
${entered_ticket_count} tickets cannot be fulfilled.
```

- The App Component should also have an input field for searching for a particular concert.
- If the user enters the concert name in the input field, only the concerts that match the input should be shown.

BACK-END: NodeJS

- Use NodeJS and MongoDB for back-end purposes.
- concert is the name of the database used in this application.
- There is a collection file namely concert.js that resides inside src/models.

Schema of the concert collection

Sl.No	Field Name	Type	Required
1	_id	ObjectID	Auto-generated
2	concert_name	String	TRUE
3	date	Date	TRUE
4	location	String	TRUE
5	available_tickets	Number	TRUE

ROUTERS

- There is a single file for routers namely concert.js that resides inside src/routers.

GET Method

- /concert -> This method should fetch the data from the concert collection.
- If the data is fetched successfully, then you should send a response code of 200.
- If something goes wrong in execution and the data fetching is unsuccessful, then you should send a response code of 400.

PATCH Method

- This method is used to update available_tickets in the concert collection for the document having _id equal to the id that comes with the request URL.

Sample request: /concert/<id>

Sample data sent with the request body:

```
{
  "tickets": 1
}
```

- Decrement the available_tickets value using the ticket count passed in the request body.
- If the decrement is completed successfully, send the response code 200 and the following message:

```
Ticket booked successfully
```

- If some validations fail and the data is not updated, then you should send a response code of 400.